PHASE 2 · DIAGRAMS · ALGORITHMS · PSEUDOCODE · STACK

Financial Statement Review Agent

Project 2 — draw these, then build. Simple and enough for the use case.

Field	Value
Document ID	FSRA-DIAG-001
Input	FSRA-BRD-001 v1.1 · FSRA-HLD-001 v1.1
Version / Date	1.1 · 8 September 2026

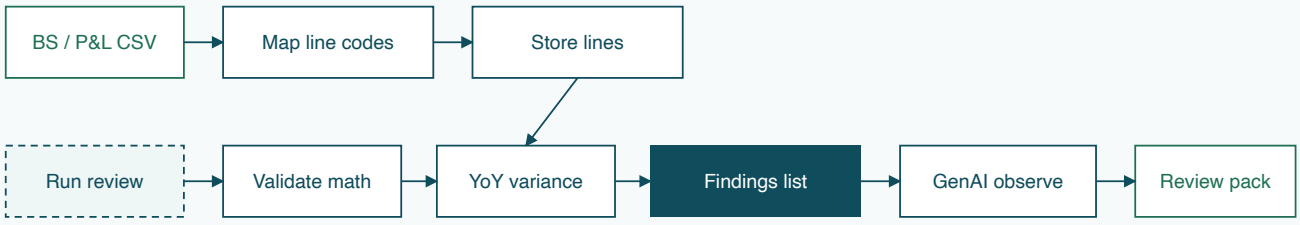
1. Use case

Figure 1 — who uses it and what they can do


```
flowchart LR
  R[Financial reviewer] --> BOT[FS Review Agent]
  L[Finance lead] --> BOT
  BOT --> U[Upload / select BS+P&L]
  BOT --> C[Run math + YoY checks]
  BOT --> F[View findings + evidence]
  BOT --> O[Read observations + export]
```

2. Architecture (how truth moves)

Figure 2 — load once, review every run




```
flowchart LR
  CSV[BS / P&L] --> MAP[Map line codes]
  MAP --> STORE[Store normalized lines]
  RUN[Run review] --> VAL[Validate math + consistency]
  VAL --> YOY[YoY variance]
  STORE --> VAL
  STORE --> YOY
  YOY --> FIND[Findings list]
  FIND --> GEN[GenAI observations from findings only]
  GEN --> PACK[Review pack to UI]
```

3. Sequence — one review

Figure 3 — controller orchestrates services

#	From → To	What happens
1	UI → Controller	POST /reviews { company, years }
2	Controller → Model	load lines for current + prior
3	Controller → ValidationSvc	math + consistency → findings
4	Controller → YoYSvc	material variances → findings
5	Controller → ObservationSvc	findings in → observations out
6	Controller → UI	review pack DTO

```
sequenceDiagram
  participant U as Reviewer UI
  participant C as Controller
  participant M as Model
  participant V as ValidationSvc
  participant Y as YoYSvc
  participant O as ObservationSvc
  U->>C: POST /reviews
  C->>M: load BS + P&L lines
  C->>V: validate(lines)
  V-->>C: findings_math
  C->>Y: compare(current, prior)
  Y-->>C: findings_yoy
  C->>O: explain(findings)
  O-->>C: observations
  C-->>U: review pack
```

4. Algorithms (simple)

A. Math check

For each defined total rule (example: Total Assets = sum of asset children):

```
expected = sum(child amounts)
actual   = reported total
if abs(expected − actual) > epsilon:
    FAIL with evidence
else:
    PASS
```

B. Consistency (BS identity)

```
assets = line(ASSETS)
liab   = line(LIABILITIES)
equity = line(EQUITY)
diff   = assets − (liab + equity)
if abs(diff) > epsilon:
    FAIL CONS-01
else:
    PASS
```

C. Year-on-year variance

```
for each mapped line present in both years:
    abs_chg = current − prior
    pct_chg = abs_chg / abs(prior)   # guard prior == 0
    if abs(pct_chg) >= THRESHOLD or abs(abs_chg) >= AMOUNT_FLOOR:
        mark MATERIAL and keep both year values as evidence
```

Default demo threshold: 20% or a small absolute floor for zero-base lines.

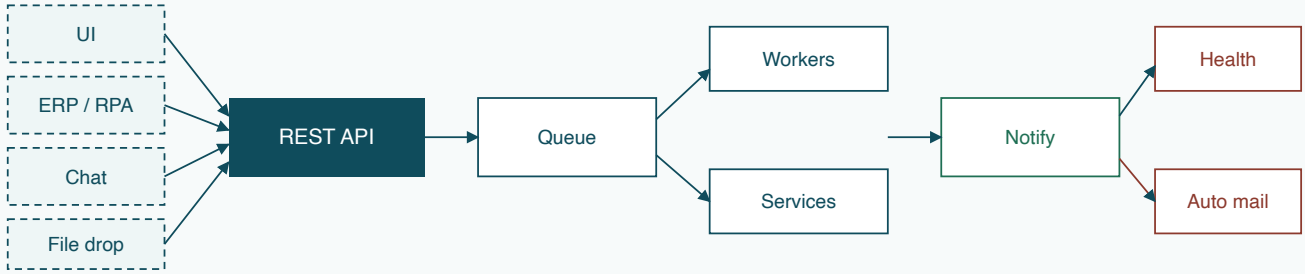
5. Pseudocode (core run)

function run_review(company_id, current_year, prior_year)

```
function run_review(company_id, current_year, prior_year):
    lines_cur  = model.load_lines(company_id, current_year)   # BS + P&L
    lines_pri  = model.load_lines(company_id, prior_year)

    findings = []
    findings += validation.validate_math(lines_cur)
    findings += validation.validate_consistency(lines_cur)
    findings += yoy.compare(lines_cur, lines_pri, threshold=0.20)

    observations = observation.write(findings)   # GenAI or template
    # observation.write MUST copy amounts from findings only

    return ReviewPack(
        summary = count(findings),
        findings = findings,
        variances = filter_material(findings),
        observations = observations
    )
```

6. Platform diagram — integrate · scale · watch

Figure 4 — one API, many doors; queue under load; alert when down




```
flowchart LR
  UI & ERP & Chat & Files --> API[REST Controller]
  API --> Q[Job queue under load]
  Q --> W[Workers: math || YoY then observe]
  W --> N[Notify: email / webhook]
  API --> H[GET /health]
  H -->|down| M[Auto mail to ops]
```

7. Stack to follow (locked for build)

Build the core first. Platform pieces are thin wrappers around the same Controller.

Layer	Use	Why
Language	Python 3	Rules + API + GenAI in one place
View	Streamlit	Upload, findings, feedback thumbs
Controller	FastAPI	REST for any environment
Services	validation / yoy / observation / notify	Truth vs wording vs alerts
Queue (scale)	In-process or Redis list	Traffic without blocking UI
Model	SQLite + JSON fixtures	Demo + tests + run audit
Health	`GET /health` + SMTP / console mailer	Down → auto mail
GenAI	OpenAI if key, else templates	Evidence-bound
Environments	.env · local / test / demo	Keys, thresholds, alert recipients
Tests	pytest MATH / CONS / YOY / health	Judge metrics
CI/CD	GitHub Actions → deploy demo	Deployment criterion

Run	Command (planned)
Load sample data	`python -m src.ingest`
API	`uvicorn api.main:app --reload --port 8000`
UI	`streamlit run ui/app.py`
Health check	`curl localhost:8000/health`
Tests	`pytest -q`

8. Folder shape (when we code)

```
financial-statement-review-agent/
  docs/                 # BRD · HLD · this file · PDFs
  api/                  # Controller + /health + webhooks
```

```
src/
  models/             # entities / repositories / audit
  services/           # validation, yoy, observation, notify
  workers/            # async pipeline (optional under load)
  config/             # environments
ui/                   # View + feedback controls
data/                 # sample BS / P&L fixtures
tests/                # rules + health + notify stubs
.github/workflows/    # CI/CD
.env.example
```

FSRA-DIAG-001 v1.1 · Phase 2 diagrams + algorithms + platform · Input: FSRA-BRD-001 / FSRA-HLD-001 v1.1 · Next: implement MVC project after these PDFs are accepted.